Modèles Algorithmiques Fondamentaux et Mécanismes d'Accès aux Données

Thème principal : Conception d'algorithmes et modèles de parcours sur structures à accès direct

Synthèse exécutive : Ce cours magistral établit la méthodologie de conception algorithmique sur les structures de données en R1.01. Avant toute écriture de code, il est nécessaire d'analyser le contrat d'interface (accès direct, séquentiel ou dynamique). Le cours détaille les trois modèles algorithmiques fondamentaux sur structures à accès direct : le parcours complet systématique, le parcours complet conditionné et la recherche de première occurrence. Il insiste sur la gestion stricte des bornes d'indices (0 à N-1), le principe du pessimisme algorithmique et l'obligation de la double condition d'arrêt dans les recherches.

1. Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

Toute démarche de conception algorithmique débute par l'analyse du contrat d'interface offert par la structure de données. Une structure à accès direct (comme un tableau statique ou une chaîne de caractères en C++) permet d'atteindre n'importe quelle case T[i] en temps constant O(1). En revanche, une structure séquentielle (comme un fichier texte ou un flux CSV) ne propose qu'un curseur se déplaçant ligne par ligne. Tenter d'accéder à la i-ème ligne d'un fichier avec la syntaxe par indice 'T[i]' est une erreur conceptuelle majeure car le système d'exploitation ne supporte pas cet accès direct.

- Accès direct : Adressage instantané T[i] en O(1)
- Accès séquentiel : Parcours élément par élément sans saut d'indice
- Contrat d'interface : Respect impératif des primitives de la structure

Cas concret enseignant : *Rédiger un algorithme avec 'T[i]' pour lire un fichier texte séquentiel en TP ou examen conduit directement à un zéro sur la question, le curseur d'un fichier ne fonctionnant qu'en mode avance-ligne.*

2. Modèle 1 : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une structure doit subir un traitement inconditionnel (ex: affichage de tout le tableau, réinitialisation à zéro, conversion en majuscules). La taille N étant connue avant l'exécution, l'algorithme utilise une boucle bornée 'Pour'. En langage C++, les indices s'étendent strictement de 0 à N-1. L'évaluation de la case T[N] constitue un dépassement de capacité mémoire (out of bounds) provoquant une erreur d'exécution.

- Exhaustivité du parcours (100% des éléments)
- Boucle bornée Pour (for)
- Plage d'indices de 0 à N-1
- Risque d'erreur 'out of bounds'

Cas concret enseignant : *Pour afficher un tableau de N éléments, écrire 'Pour i allant de 0 à N' est la faute la plus fréquente chez les débutants. Cela provoque un plantage 'Segmentation Fault' lors de l'accès à la case T[N].*

3. Modèle 2 : Le Parcours Complet avec Traitement Conditionnel

Le parcours complet conditionné explore la totalité de la structure mais n'applique le traitement qu'aux éléments répondant à un prédicat logique (ex: compter les nombres pairs, sommer les valeurs positives). La structure associe une boucle bornée 'Pour' et une alternative 'Si'. Attention particulière : si le prédicat compare l'élément courant T[i] à son voisin précédent T[i-1], la boucle doit impérativement débuter à i = 1 pour éviter l'évaluation de T[-1].

- Filtrage et inspection globale

- Structure alternative Si (if) imbriquée
- Ajustement des bornes en cas de comparaison contiguë

Cas concret enseignant : *Filtrage d'un tableau d'entiers pour n'en conserver et afficher que les éléments strictement positifs.*

4. Modèle 3 : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux parcours complets, la recherche s'interrompt dès que l'élément cible est identifié. La conception repose sur le pessimisme algorithmique : le drapeau 'trouve' est initialisé à Faux. La boucle itérative 'Tant que' exige obligatoirement une double condition d'arrêt : la recherche s'arrête soit si l'élément est trouvé, soit si la fin de la structure est atteinte (i >= N). Omettre l'une des deux conditions entraîne une boucle infinie ou une violation d'accès mémoire.

- Pessimisme algorithmique (trouve = Faux)
- Boucle non bornée Tant que (while)
- Double condition d'arrêt obligatoire (Succès OU Épuisement)
- Variantes par rupture d'indice ou test en tête

Cas concret enseignant : *Règle absolue du professeur : 'En informatique, soyez pessimistes ! Partez du principe que la valeur n'existe pas.' Proposer un algorithme de recherche avec une seule condition d'arrêt à l'examen vaut zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systematic sur une structure à accès dynamique ?
Réponse du professeur : Oui, à condition que la structure dynamique fournisse le moyen de connaître sa taille ou d'accéder au premier et au dernier élément, mais l'implémentation utilisera le mécanisme d'accès propre à cette structure (comme des pointeurs ou itérateurs).

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La variante 1 conserve exactement l'indice où l'élément a été trouvé dès la sortie de boucle. La variante 2, avec son test unique en tête de boucle Tant que, incrémente l'indice une dernière fois avant de sortir, ce qui nécessite un réajustement d'indice après la boucle.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionnel dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut impérativement ajuster la borne de démarrage de la boucle à i = 1 au lieu de i = 0 pour éviter d'évaluer la case T[-1] qui est hors de la mémoire allouée.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il faut utiliser le Modèle 2 (Parcours complet avec traitement conditionné) et non une recherche, car il faut continuer d'explorer l'intégralité du tableau même après avoir trouvé le premier '5'.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme C++ complet (norme IUT d'Anglet) qui parcourt un tableau d'entiers et affiche uniquement ses éléments strictement positifs.

Démarche de résolution :
1. Identification du modèle : Parcours complet avec traitement conditionné (Modèle 2). 2. Détermination des bornes d'indice : Indice initial i = 0, Indice final i = TAILLE - 1. 3. Structure de contrôle : Boucle bornée 'for' de i = 0 à i < TAILLE. 4. Condition d'évaluation : Structure alternative 'if (tableau[i] > 0)' pour filtrer la sortie.

```cpp
/*
  Nom du fichier : afficher_positifs.cpp
  Auteur         : Matéi de Saint Léger
  Date           : 13/11/2026
  Objectif       : Parcours complet conditionné d'un tableau d'entiers
*/

#include <iostream>
using namespace std;

int main()
{
    // CONSTANTES
    const int TAILLE = 5;

    // VARIABLES
    int tableau[TAILLE] = {-3, 5, 0, 12, -8};
    int i; // Indice de parcours

    // TRAITEMENTS
    cout << "Éléments strictement positifs :" << endl;

    for (i = 0; i < TAILLE; i++)
    {
        if (tableau[i] > 0)
        {
            // (écran) << afficher << tableau[i]
            cout << "-[ " << tableau[i] << " ]" << endl;
        }
    }

    return 0;
}
```

Résultat : Le programme parcourt exhaustivement le tableau et affiche uniquement '5' et '12' sans erreur de dépassement de capacité d'indice.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique devez-vous utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : Le Modèle 3 (Recherche de première occurrence), car le parcours s'arrête dès que la valeur 10 est rencontrée.

Q : Quel est le risque critique si l'on écrit une boucle 'Pour i allant de 0 à N' sur un tableau de N éléments indicé à partir de 0 ?

R : Une erreur d'accès mémoire 'out of bounds' (Segmentation Fault) en tentant de lire la case T[N] qui est hors du tableau.

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'un algorithme de recherche de première occurrence et quelles sont-elles ?

R : Elle doit comporter deux conditions d'arrêt : 1) Le succès de la recherche (élément trouvé), 2) L'épuisement de la structure (fin du tableau atteinte).